DSA pseudo codes

1. BFS

```
void bfs(List<List<Integer>> graph,
         int start,
         int vertices) {

    boolean[] visited =
            new boolean[vertices];

    Queue<Integer> queue =
      new LinkedList<>();

    visited[start] = true;

    queue.add(start);

    while (!queue.isEmpty()) {

        int node = queue.poll();

        for (int neighbor :
             graph.get(node)) {

            if (!visited[neighbor]) {

                visited[neighbor] = true;

                queue.add(neighbor);
            }
        }
    }
}
```

2. DFS

```
void dfs(List<List<Integer>> graph,
        int node,
        boolean[] visited) {

    visited[node] = true;

    for (int neighbor :
        graph.get(node)) {

        if (!visited[neighbor]) {

            dfs(graph,
                neighbor,
                visited);
        }
    }
}
```

3. Prim's

```
void prim(int graph[][], int n) {

    boolean visited[] = new boolean[n];

    visited[0] = true;

    for (int count = 0; count < n - 1; count++) {

        int min = Integer.MAX_VALUE;
        int x = 0, y = 0;

        for (int i = 0; i < n; i++) {

            if (visited[i]) {

                for (int j = 0; j < n; j++) {

                    if (!visited[j] &&
                        graph[i][j] != 0 &&
                        graph[i][j] < min) {

                        min = graph[i][j];
                        x = i;
                        y = j;
                    }
                }
            }
        }

        visited[y] = true;
    }
}
```

Kruskals's

```
void kruskal(int edges[][], int e) {

    for (int i = 0; i < e - 1; i++) {

        for (int j = i + 1; j < e; j++) {

            if (edges[i][2] > edges[j][2]) {

                int temp[] = edges[i];
                edges[i] = edges[j];
                edges[j] = temp;
            }
        }
    }

    for (int i = 0; i < e; i++) {

        int u = edges[i][0];
        int v = edges[i][1];

        // if no cycle
        // include edge
    }
}
```

Hash Table (linear)

```
void insert(int table[], int size, int key) {

    int index = key % size;

    while (table[index] != 0) {

        index = (index + 1) % size;
    }

    table[index] = key;
}
```

Hashing (Quadratic)

```
void insert(int table[], int size, int key) {

    int index = key % size;

    int i = 1;

    while (table[index] != 0) {

        index = (key + i * i) % size;

        i++;
    }

    table[index] = key;
}
```

Hashing (Double)

```
void insert(int table[], int size, int key) {

    int index = key % size;

    int step = 7 - (key % 7);

    while (table[index] != 0) {

        index = (index + step) % size;
    }

    table[index] = key;
}
```

Dijkstra

```
void dijkstra(int graph[][], int n, int src) {

    int dist[] = new int[n];
    boolean visited[] = new boolean[n];

    for (int i = 0; i < n; i++) {

        dist[i] = Integer.MAX_VALUE;
        visited[i] = false;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {

        int u = -1;
        int min = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {

            if (!visited[i] && dist[i] < min) {

                min = dist[i];
                u = i;
            }
        }

        visited[u] = true;

        for (int v = 0; v < n; v++) {

            if (!visited[v] &&
                graph[u][v] != 0 &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}
```